

Name:
Z number:

Total Points: 15

COP 3410-001: Data Strc Algrthm Anlys Python

Florida Atlantic University

Recursion Worksheet

NOTE:

- Each question carries 1 point.
- All solutions must be answered in the space labeled “put your solutions here.”
- submit the completed worksheet in the form of a PDF.
- The file name must be *YourName_Recursion Worksheet.pdf*

1. Which data structure is responsible for keeping track of function calls during recursion? choose the correct option. and justify your answer.
 - a. Queue
 - b. Stack
 - c. LinkedList
 - d. Graph

Put Your Option Here:

Justification:

Answer:b

when a recursive function is called its execution is pushed onto the stack. When the base case is reached the functions return and are popped off the srack in the reverse order they were called

2. What would happen if the base case is not defined in the recursive method? choose the correct options from the below. and justify your answer.
- a. Stack Overflow
 - b. Stack Underflow
 - c. Program Crashes
 - d. Infinite Loop

Put Your option(s) Here:

Justification:

Answer: a

Without a base case, the function will enter a logical infinite loop of recursive calls. However, because each call consumes finite memory on the call stack, the program will rapidly run out of memory space, resulting in a stack overflow error that causes the program to crash.

3. what will be the output of the following program? Show recursive trace for the fun(3)?
what will be the output of print(fun(5))?

```
def fun(n):  
    if n == 4:  
        return n  
    else:  
        return 2 * fun(n + 1)  
  
print(fun(3))
```

Put Your Solution Here:

Output: 8

Fun(3) is called

Returns $2 * \text{fun}(3+1)$

Fun(4) is called returns 4

Previous call solves $2 * 4$ resulting in 8

Function increments n until base case of 4 is reached. Then doubled

4. Given a sorted list with indices ranging from 0 to 50, and assuming there is a recursive binary search function called `search()`, what would be the sequence of function calls made by `search()` when searching for an item located at index 6? The initial function call is `search(0, 50)`.
choose the correct option. and justify your answer.
- a. `search(0, 25) search(0, 12) search (0, 6)`
 - b. `search(0, 25) search(0, 12)`
 - c. `search (0, 25) search (0, 12) search (0, 6) search (0, 3)`

Put Your Solution Here:

Justification:

Answer: b

1st recursive call `search(0, 25)` midpoint 12

2nd `search (0,12)` midpoint 6

Since the target 6 perfectly matches the new mdipoint index 6 the algorithm finds the item and returns immediately without needing to make another recursive call

5. Which searching algorithm works best for the following array:
arr = [18,23,11,9,365,11]
choose the correct option. and justify your answer.
- a. Binary Search
 - b. Linear Search

Put Your Solution Here:

Justification:

B

Binary usually requires a sorted dataset, linear is best here

-
6. What is returned as the output due to the call fun(2,9)?

```
def fun(x, y):  
    if y == x:  
        return x  
    elif y > x:  
        return fun(x, y - x)  
    else:  
        return fun(x - y, y)
```

Put Your Solution Here:

Answer: 1

7. Will fun(50) iterate infinitely? What is returned as a result of the call fun(50)?

```
def fun(y):  
    if y == 0:  
        return y-5  
    else:  
        return fun(y-10)
```

Put Your Solution Here:

Will not iterate infinitely, result is -5

-
8. What is the output of the following function code?

```
def fun(n):  
    if (n > 100):  
        return n - 10  
    return fun(fun(n + 11))  
print(fun(96))
```

Put Your Solution Here:

9. Write a Program to find the length of a string using recursion. Ask for the user input to test the code.

Put Your Solution Here:

```
def string_length(s):  
    if s == "":  
        return 0  
    else:  
        return 1 + string_length(s[1:])  
  
user_input = input("Enter a string: ")  
print("Length:", string_length(user_input))
```

10. Write a recursive function named zigzag that returns a string of n characters as follows.
The middle character (or the middle two characters if n is even) is an asterisk (*). All characters before the asterisks are '<'. All characters after are '>'. Report an error if n is not positive. Ask for the user input to test the code.
Example: n =6, pattern is <<*>>
if n=5 pattern is <<*>>

Put Your Solution Here:

```
def zigzag(n):  
    if n <= 0:  
        return "Error: n must be positive."  
    elif n == 1:  
        return "*"   
    elif n == 2:  
        return "***"  
    else:  
        return "<" + zigzag(n - 2) + ">"
```

```
user_input = int(input("Enter a positive number: "))  
print(zigzag(user_input))
```


11. In a sorted array containing prime numbers from 2 to 311, approximately how many elements would a binary search algorithm need to examine before determining that 52 is not present in the array, indicating that 52 is not a prime number?

Put Your Solution Here:

Explanation:

6 to 7 elements

There are 64 prime numbers between 2 and 311. each steps halves the amount
About 7 examinations before determining 52 is not in the array

12. What is the Big O notation for the time complexity of the calculate_power function in terms of the input parameters base_number and exp? Explain? You can assume that multiplying two numbers takes time $O(1)$.

```
def calculate_power(base_number, exp):
    if exp == 0:
        return 1
    elif exp % 2 == 0:
        power = calculate_power(base_number, exp // 2)
        return power * power
    else:
        power = calculate_power(base_number, exp // 2)
        return base_number * power * power
```

Put Your Solution Here:

Explanation:

$O(\log \exp)$

function uses exponentiation by squaring.

Each step halves the problem size. This yields a logarithmic time complexity

13. Let's consider a recursive function g , where $g(0)$ is defined as 1 and $g(n)$ is defined as $2 * g(n-1) + n$. Calculate the value of $g(5)$ by writing the step-by-step process.

Put Your Solution Here:

14. How many times does the below function recur? Write the recursive trace of this program

```
def func(x):  
    if(x>20):  
        return 0  
    if(x%2):  
        return 1+func(x+1)  
    return 1+func(x+3)  
print(func(1))
```

Put your Solution here:

Functions makes 11 calls

F

15. Understand the code below, write the following code's output, and explain how this output is achieved.

```
LIMIT = 1000
def fun2(n):
    if (n <= 0):
        return
    if (n > LIMIT):
        return
    print(n, end=" ")
    fun2(2 * n)
    print(n, end=" ")

print(fun2(7))
```

Put your solution here:

Explanation:

7 14 28 56 112 244 448 896 896 448 244
112 56 28 14 7 none

The function prints n then recursively calls itself with $2*n$. It doubles and prints the number until it equates to 1792.

Function executes second `print(n)` statements printing in reverse. The wrapper prints the final return value of the initial call.